Whitebox Learning, Inc.

MLOps

Fraud Detection System (XYZ Corp)

STUDY GUIDE

Sampath Velupula

Whitebox Learning, Inc. Teams

GitHub Repository: mlops-fraud-detection-use-case
Role: MLOps Engineer / Machine Learning Engineer

Contents

1. Business Problem & Objectives 2
1.1 The Business Problem 2
1.2 Current Operational Challenges 3
1.3 Business Objectives 3
2. Existing Fraud Detection Landscape 4
2.1 Limitations of Current Rule-Based Systems 4
3. Solution Evaluation and Architecture Selection 4
3.1 Solution 1: Rules-Only Systems 5
3.2 Solution 2: Black Box Vendor Models 5
3.3 Solution 3: Custom Machine Learning with MLOps (Selected) 5
4. Proof of Concept (POC) 6
4.1 How the POC Started 6
4.2 What Was Built 6
4.3 Technologies Used 7
4.4 POC Learnings 7
5. Phase One Production System 7
5.1 Critical System Requirements 8
5.2 High-Level Architecture 8
5.3 Design and Implementation Overview 9
6. Data Ingestion and Data Versioning 9
6.1 Data Sources 9
6.2 DVC Based Data Versioning 10
6.2.1 Why We Did That 10
6.2.2 Workflow 10
6.3 Challenges Faced 10

© Whitebox Learning, Inc. 2026

7. Feature Engineering and Feature Store 11
7.1 Feature Categories 11
7.2 Feature Store Architecture 11
7.2.1 Why We Did That 12
7.3 Online and Offline Consistency 12
7.4 Challenges Faced 12
8. Model Development (Phase One) 12
8.1 Algorithm Selection 13
8.2 Logistic Regression Design 13
8.3 Training Strategy 14
8.4 Model Validation 14
9. Experiment Tracking and Model Registry 14
9.1 MLflow Architecture 15
9.2 Experiment Design 15
9.3 Model Promotion Workflow 15
10. Model Serving and APIs 16
10.1 FastAPI Inference Service 16
10.1.1 Why We Did That 16
10.2 Request Validation and Schemas 16
10.3 Response Design 17
11. CI/CD for ML Systems 17
11.1 Training Pipelines 17
11.2 Deployment Pipelines 17
12. Monitoring, Observability, and Drift Detection 18
12.1 Data Drift 18
12.2 Feature Drift 18

© Whitebox Learning, Inc. 2026

12.3 Model Performance Drift 19
12.4 System Metrics 19
13. Governance, Security, and Compliance 19
13.1 Explainability 19
13.2 Audit Trails 19
13.3 Access Control 20
14. Deployment on AWS 20
14.1 Infrastructure Overview 20
14.2 Containerization and Orchestration 21
14.3 Environment Strategy 21
15. Phase Two: Advanced Modeling (Future State) 21
15.1 Motivation for Phase Two 21
15.2 XGBoost Model Design 21
15.3 Comparison with Logistic Regression 22
16. Future Enhancements 22
17. References 22

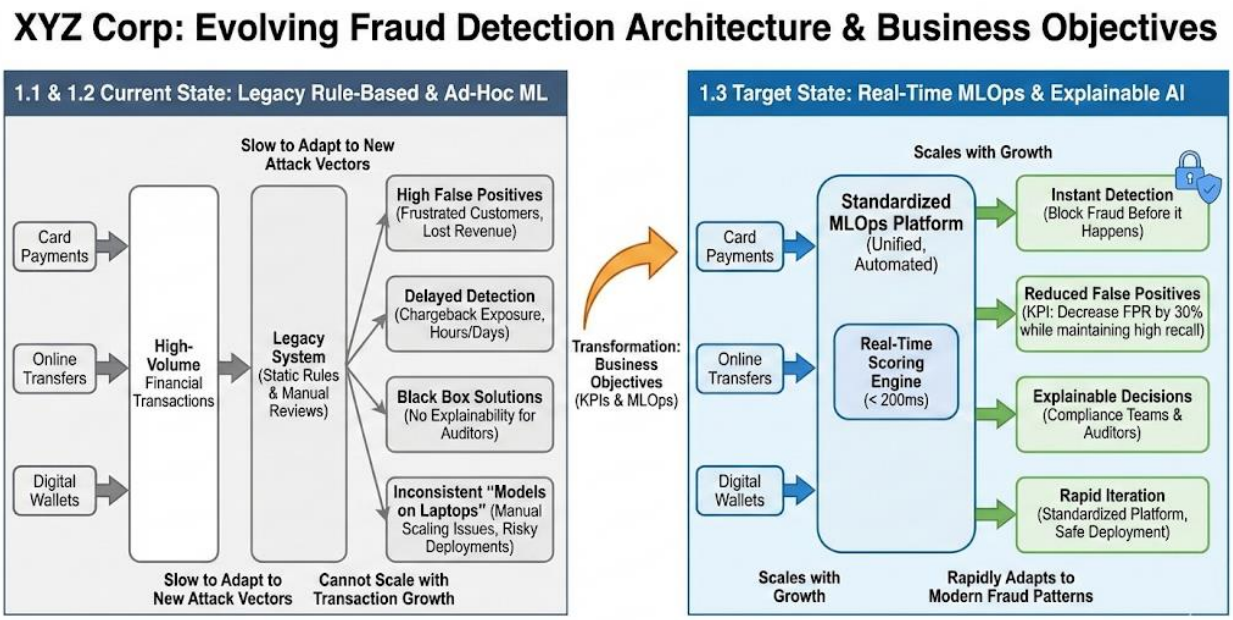
© Whitebox Learning, Inc. 2026

1. Business Problem & Objectives

1.1 The Business Problem

XYZ Corp processes millions of high-volume financial transactions daily across card payments, online transfers, and digital wallets. While fraudulent activity represents only a small fraction of these transactions, it creates outsized financial losses, significant customer dissatisfaction, and regulatory exposure.

Fraud patterns are not static; they evolve rapidly and differ across regions, channels, and customer segments. The legacy approach relied on static rules and manual reviews, which failed to adapt quickly enough to new attack vectors and could not scale with the company's transaction growth.



Addressing High-Volume Fraud with Real-Time Detection, Reduced False Positives, and Compliance

© Whitebox Learning, Inc. 2026

1.2 Current Operational Challenges

- **High False Positive Rates:** The existing rule-based system flags too many legitimate transactions, leading to frustrated customers and lost revenue.
- **Manual Scaling Issues:** Rule tuning is manual, reactive, and cannot keep pace with the speed of modern fraud.
- **Delayed Detection:** Fraud is often detected hours or days later, increasing chargeback exposure.
- **Lack of Explainability:** Vendor "black box" solutions provided scores without reasons, making it impossible for compliance teams to audit decisions.
- **Inconsistent Deployment:** "Models on laptops" led to a lack of reproducibility and risky, manual deployments.

1.3 Business Objectives

- **Real-Time Detection:** Score transactions in near real-time (< 200ms) to block fraud before it happens.
- **Reduce False Positives (KPI):** Decrease the False Positive Rate (FPR) by 30% while maintaining high fraud recall.
- **Compliance & Explainability:** Ensure every decision is explainable to compliance teams and auditors.
- **Rapid Iteration:** Enable data scientists to experiment rapidly and deploy safely using a standardized platform.
- **Standardized MLOps:** Establish a unified platform for all risk models, moving away from ad-hoc scripts.

© Whitebox Learning, Inc. 2026

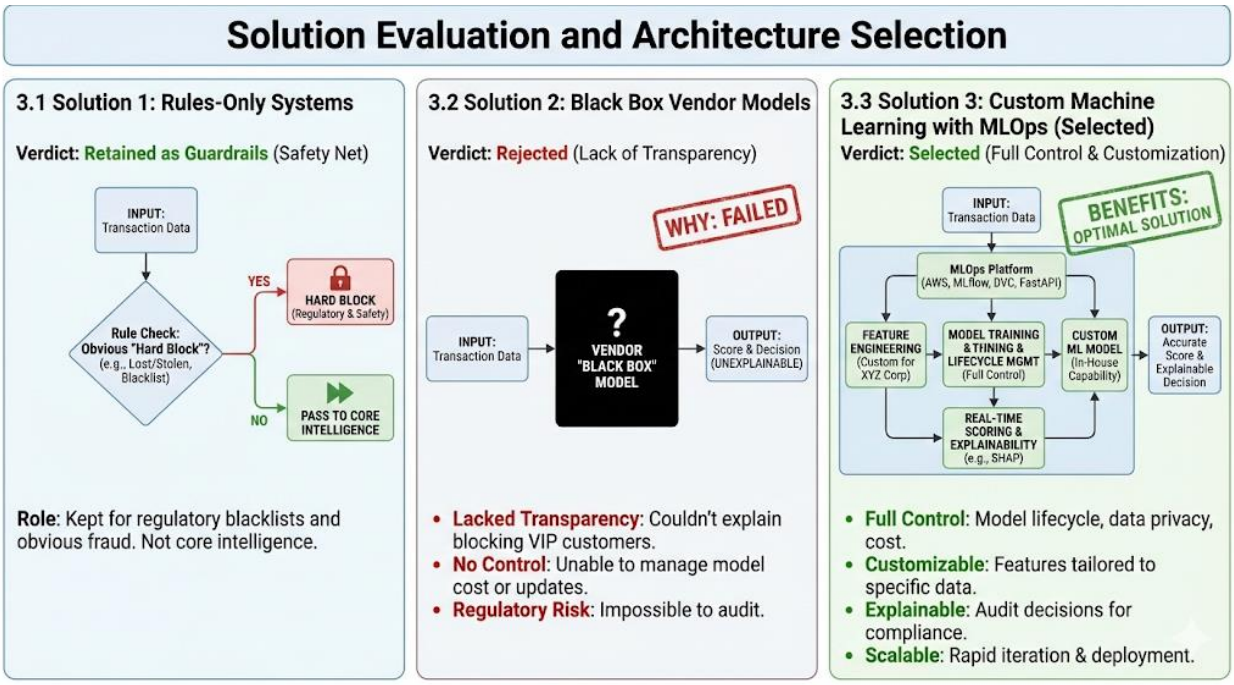
2. Existing Fraud Detection Landscape

2.1 Limitations of Current Rule-Based Systems

The legacy system relied heavily on static logic (e.g., IF amount > $5000 AND location != home THEN BLOCK).

- **Fragility:** Rules are brittle. A legitimate user traveling for business triggers the same rule as a thief.
- **Maintenance Nightmare:** As fraud evolves, the rule list grows indefinitely, becoming a tangled mess of conflicting logic that is impossible to maintain.
- **No Generalization:** Rules cannot detect novel fraud patterns that haven't been explicitly programmed.

3. Solution Evaluation and Architecture Selection



© Whitebox Learning, Inc. 2026

3.1 Solution 1: Rules-Only Systems

- **Verdict: Retained as Guardrails.**
- **Role:** We kept rules only for obvious "hard blocks" (e.g., card reported lost/stolen) and regulatory blacklists. They serve as a safety net, not the core intelligence.

3.2 Solution 2: Black Box Vendor Models

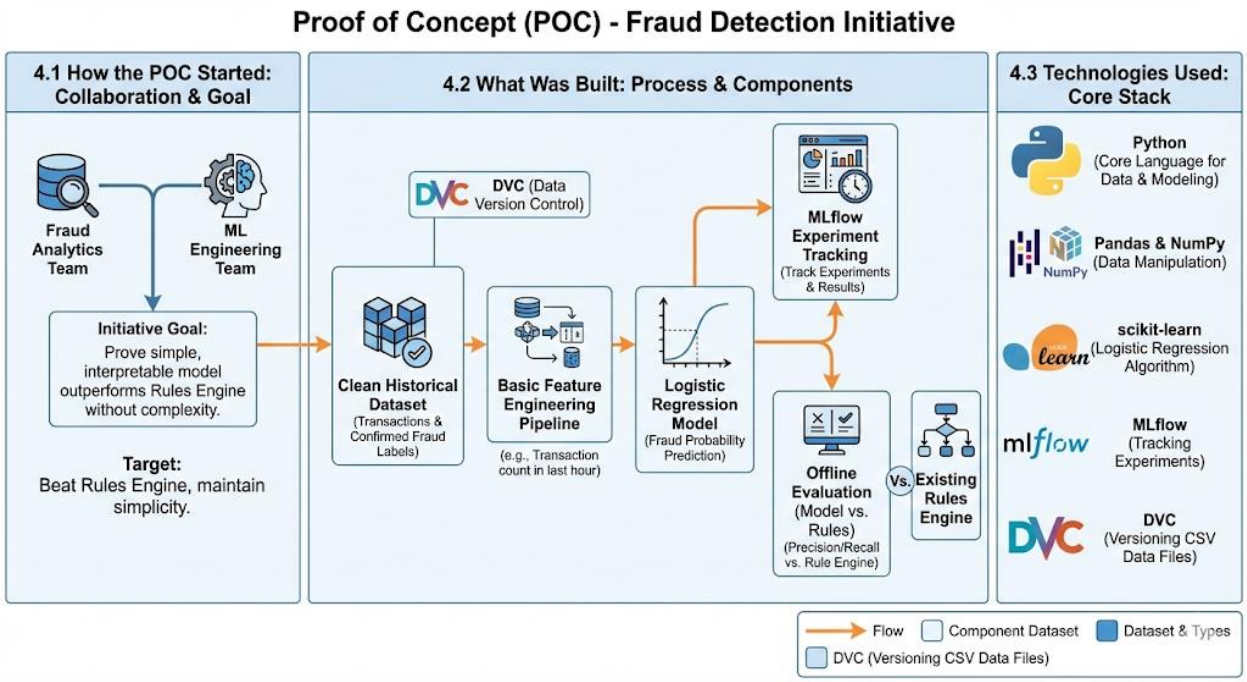
- **Verdict: Rejected.**
- **Why:** Vendor models lacked transparency. We could not explain *why* a VIP customer was blocked, and we had no control over the model's cost or update schedule.

3.3 Solution 3: Custom Machine Learning with MLOps (Selected)

- **Verdict: Selected.**
- **Approach:** Build an in-house ML capability using open-source standards (MLflow, DVC, FastAPI) on AWS.
- **Benefits:** Full control over the model lifecycle, data privacy, cost, and the ability to customize features for XYZ Corp's specific transaction data.

© Whitebox Learning, Inc. 2026

4. Proof of Concept (POC)



4.1 How the POC Started

The initiative began with a collaboration between the Fraud Analytics and ML Engineering teams. The goal was to prove that a simple, interpretable model could outperform the existing rules engine without introducing massive infrastructure complexity.

4.2 What Was Built

- A clean dataset of historical transactions with confirmed fraud labels (chargebacks).
- A basic feature engineering pipeline to calculate simple metrics (e.g., transaction count in the last hour).
- A **Logistic Regression** model to predict fraud probability.
- **MLflow** integration to track experiments and results.

© Whitebox Learning, Inc. 2026

- Offline evaluation comparing Model Precision/Recall vs. Rule Engine performance.

4.3 Technologies Used

- **Python:** The core language for data and modeling.
- **Pandas & NumPy:** For data manipulation.
- **scikit-learn:** For the Logistic Regression algorithm.
- **MLflow:** For tracking experiments.
- **DVC:** For versioning the CSV data files.

4.4 POC Learnings

- **Data Quality > Model Complexity:** Cleaning the labels and fixing timestamp issues improved performance more than trying complex algorithms.
- **Baseline Power:** Logistic Regression provided a surprisingly strong baseline that was easy to explain to stakeholders.
- **Tracking Matters:** Using MLflow immediately solved the "which script generated this result?" problem.

5. Phase One Production System

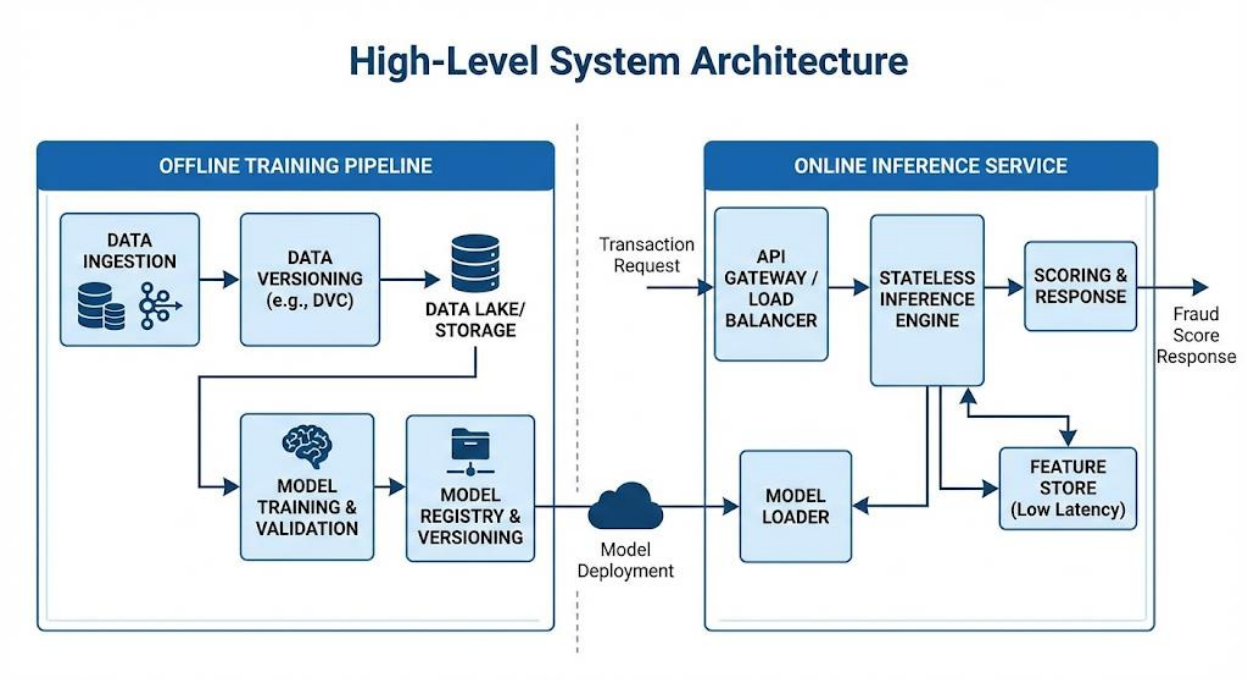
5.1 Critical System Requirements

- **Latency:** Inference must complete in under **100ms** (excluding network overhead).
- **Determinism:** The same input must always produce the same score and explanation.
- **Reproducibility:** Any deployed model must be fully reproducible from code + data.

© Whitebox Learning, Inc. 2026

- **Lineage:** Full traceability from a prediction back to the training data version.
- **Safety:** Capability to rollback to a previous model version instantly.

5.2 High-Level Architecture



The system is decoupled into two primary distinct pipelines:

1. **Offline Training Pipeline:** Handles data ingestion, versioning, training, and registry.
2. **Online Inference Service:** A stateless API that fetches features, loads the model, and scores transactions.

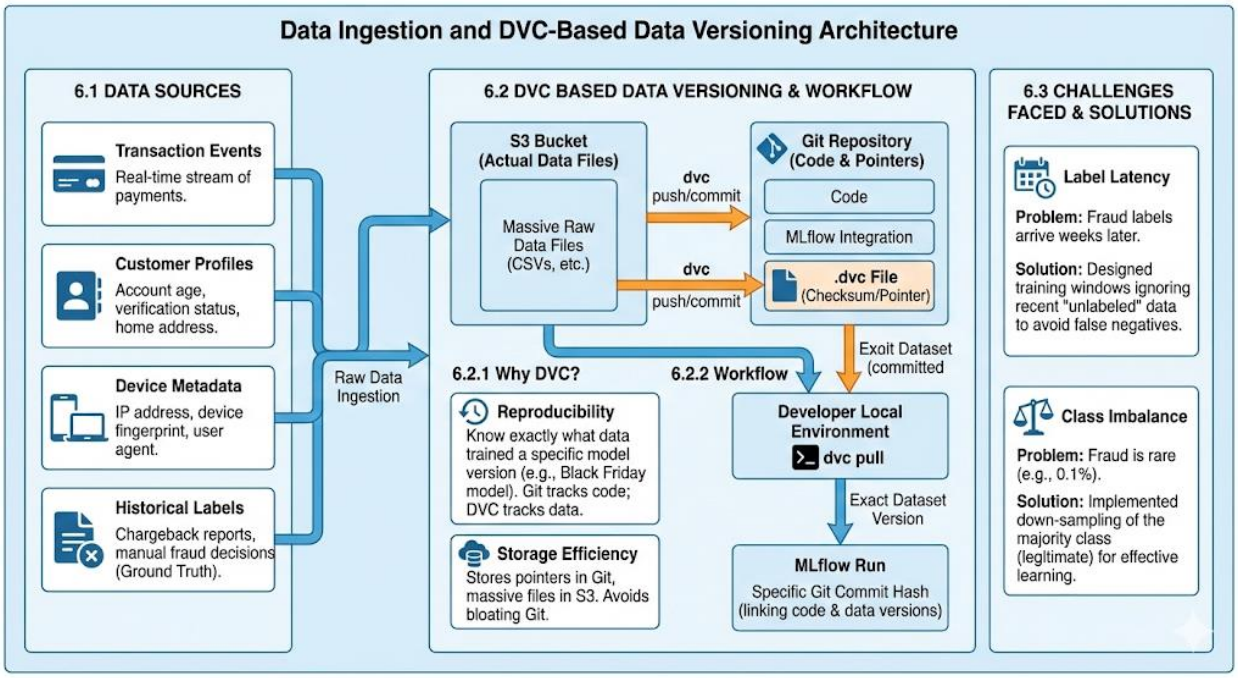
5.3 Design and Implementation Overview

- **Separation of Concerns:** Training is batch-oriented and resource-intensive. Inference is real-time and latency-sensitive. They run on separate infrastructure.

© Whitebox Learning, Inc. 2026

- **Shared Feature Definitions:** Critical to preventing "training-serving skew," feature transformations are standardized.

6. Data Ingestion and Data Versioning



6.1 Data Sources

- **Transaction Events:** Real-time stream of payments.
- **Customer Profiles:** Account age, verification status, home address.
- **Device Metadata:** IP address, device fingerprint, user agent.
- **Historical Labels:** Chargeback reports and manual fraud review decisions (the ground truth).

© Whitebox Learning, Inc. 2026

6.2 DVC Based Data Versioning

6.2.1 Why We Did That

- **Reproducibility:** We need to know *exactly* what data trained the model that was live on Black Friday. Git tracks code; DVC tracks data.
- **Storage Efficiency:** Storing massive CSVs in Git is impossible. DVC stores pointers in Git and the actual files in S3.

6.2.2 Workflow

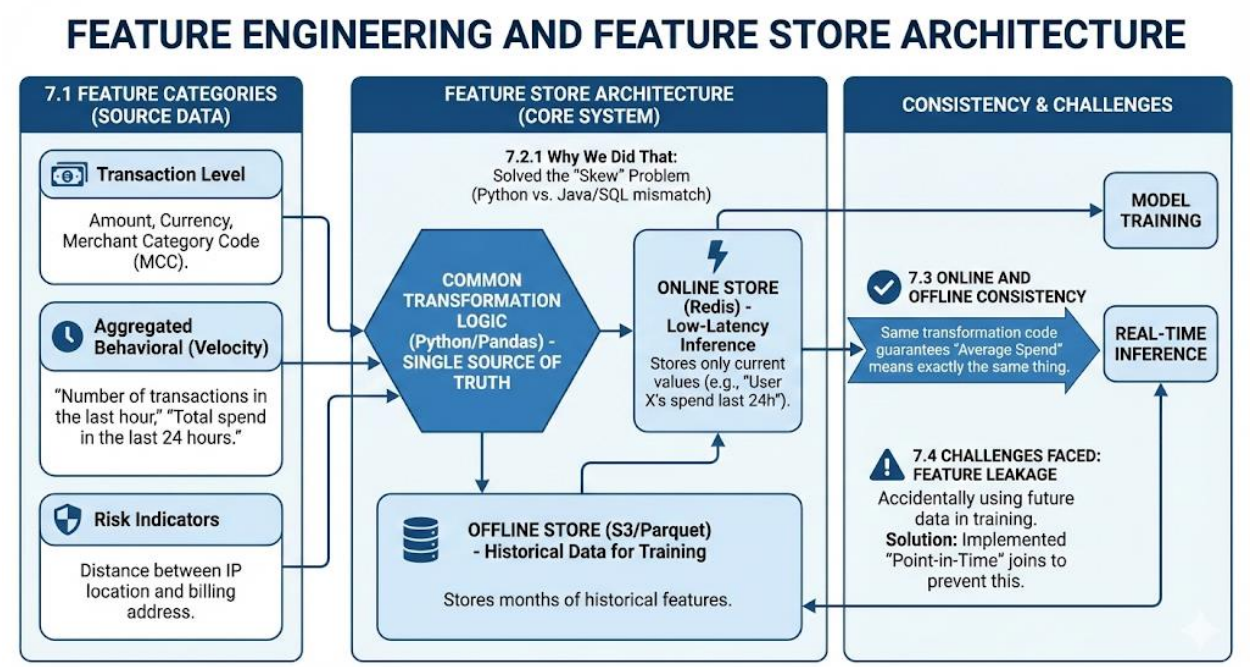
1. Raw data is dumped to an **S3 Bucket**.
2. A .dvc file (checksum/pointer) is created and committed to **Git**.
3. Developers use dvc pull to download the exact dataset version to their local environment.
4. Every MLflow run logs the specific Git commit hash, linking code and data versions.

6.3 Challenges Faced

- **Label Latency:** Fraud labels (chargebacks) often arrive weeks later. We had to design training windows that ignored recent "unlabeled" data to avoid treating undetected fraud as legitimate (false negatives).
- **Class Imbalance:** Fraud is rare (e.g., 0.1%). We had to implement down-sampling of the majority class (legitimate) to help the model learn effectively.

© Whitebox Learning, Inc. 2026

7. Feature Engineering and Feature Store



7.1 Feature Categories

- **Transaction Level:** Amount, currency, merchant category code (MCC).
- **Aggregated Behavioral (Velocity):** "Number of transactions in the last hour," "Total spend in the last 24 hours."
- **Risk Indicators:** Distance between IP location and billing address.

7.2 Feature Store Architecture

7.2.1 Why We Did That

- **The "Skew" Problem:** Data scientists wrote feature code in Python/Pandas. Engineers rewrote it in Java/SQL for production. The logic never matched perfectly.
- **Solution:** A Feature Store acts as the single source of truth for feature logic.

© Whitebox Learning, Inc. 2026

- **Offline Store (S3/Parquet):** Stores months of historical features for training.
- **Online Store (Redis):** Stores only the *current* values (e.g., "User X's spend last 24h") for low-latency retrieval during inference.

7.3 Online and Offline Consistency

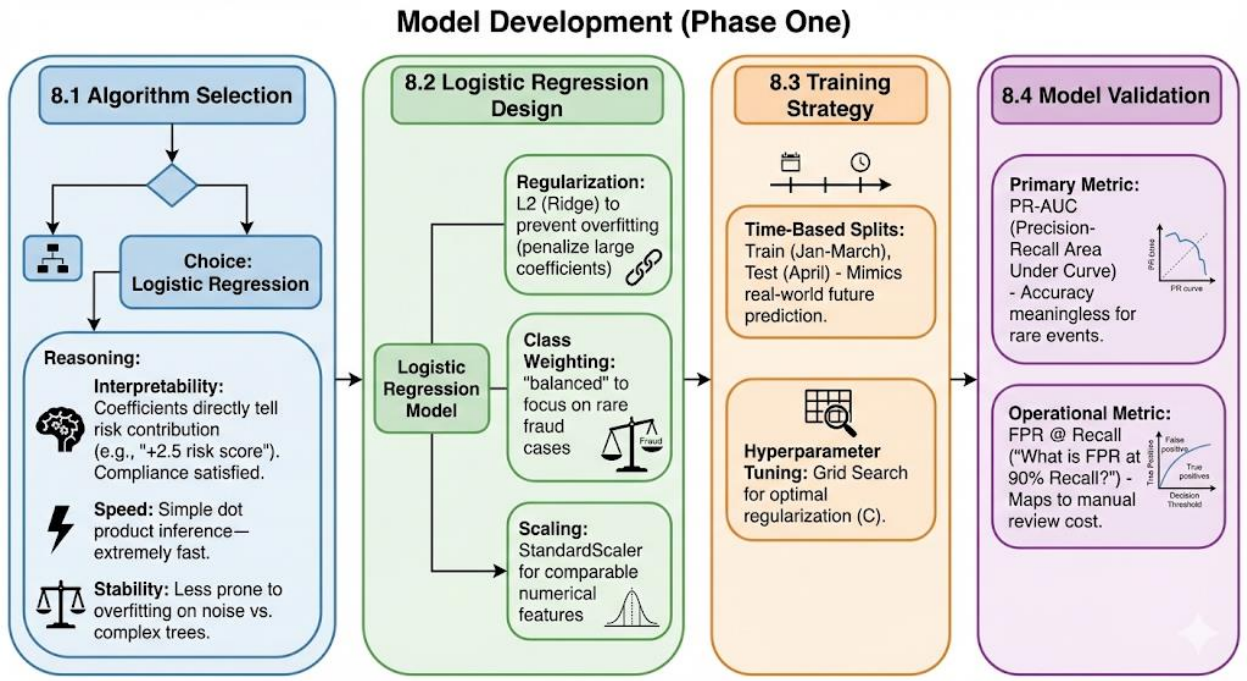
We enforced a rule where the same transformation code populates both stores. This guarantees that "Average Spend" means exactly the same thing during training and inference.

7.4 Challenges Faced

- **Feature Leakage:** Accidnetally using future data in training (e.g., calculating "total spend today" including the current transaction being predicted). We implemented "Point-in-Time" joins to prevent this.

© Whitebox Learning, Inc. 2026

8. Model Development (Phase One)



8.1 Algorithm Selection

- **Choice: Logistic Regression.**
- **Reasoning:**
  - **Interpretability:** Coefficients directly tell us the risk contribution (e.g., "High velocity adds +2.5 to the risk score"). This satisfied the Compliance team.
  - **Speed:** Inference is just a simple dot product calculation—extremely fast.
  - **Stability:** Less prone to overfitting on noise compared to complex tree models.

8.2 Logistic Regression Design

- **Regularization:** Applied **L2 Regularization** (Ridge) to prevent overfitting by penalizing large coefficients.

© Whitebox Learning, Inc. 2026

- **Class Weighting:** Applied class_weight='balanced' to force the model to pay more attention to the rare fraud cases.
- **Scaling:** All numerical features were standardized (StandardScaler) to ensure coefficients were comparable.

8.3 Training Strategy

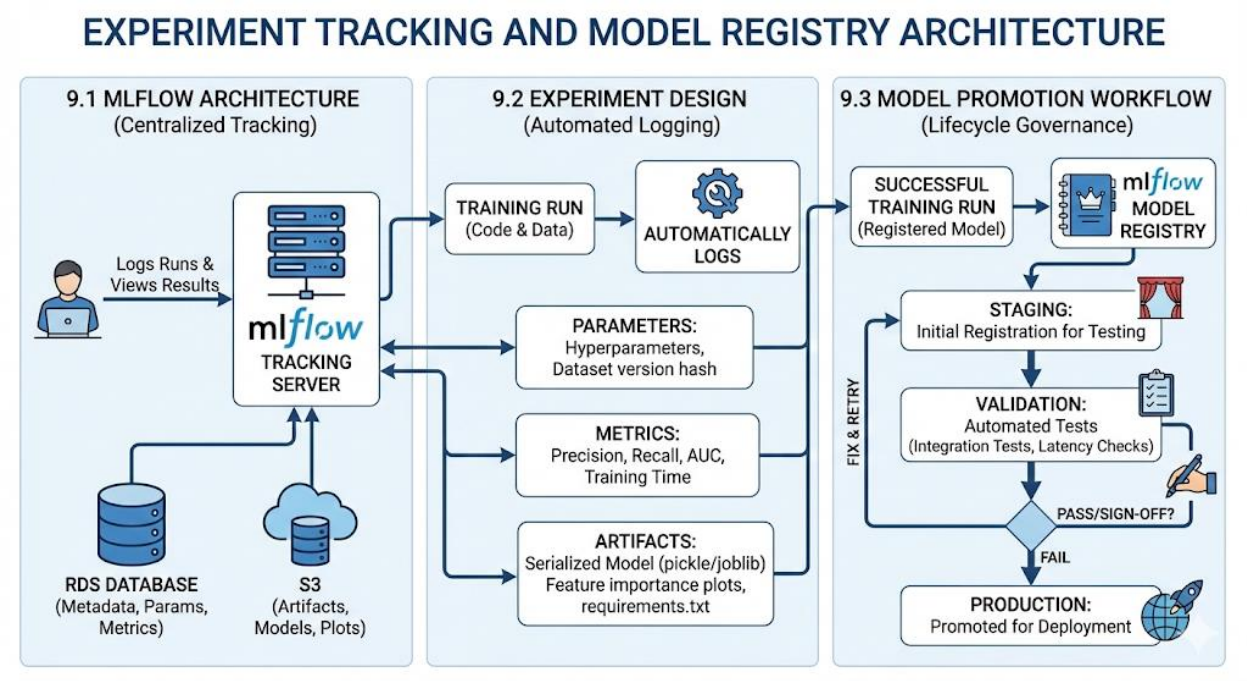
- **Time-Based Splits:** Instead of random splits (which leak future information), we trained on Jan-March and tested on April. This mimics the real-world scenario of predicting the future.
- **Hyperparameter Tuning:** Used Grid Search to find the optimal regularization strength (C).

8.4 Model Validation

- **Primary Metric: PR-AUC** (Precision-Recall Area Under Curve). Accuracy is meaningless when 99.9% of transactions are legitimate.
- **Operational Metric: FPR @ Recall:** "What is our False Positive Rate when we catch 90% of fraud?" This directly maps to the cost of manual review.

© Whitebox Learning, Inc. 2026

9. Experiment Tracking and Model Registry



9.1 MLflow Architecture

We deployed a central **MLflow Tracking Server** backed by an RDS database (for metadata) and S3 (for artifacts).

9.2 Experiment Design

Every training run automatically logs:

- **Parameters:** Hyperparameters, dataset version hash.
- **Metrics:** Precision, Recall, AUC, Training Time.
- **Artifacts:** The serialized model (pickle/joblib), feature importance plots, and a requirements.txt file.

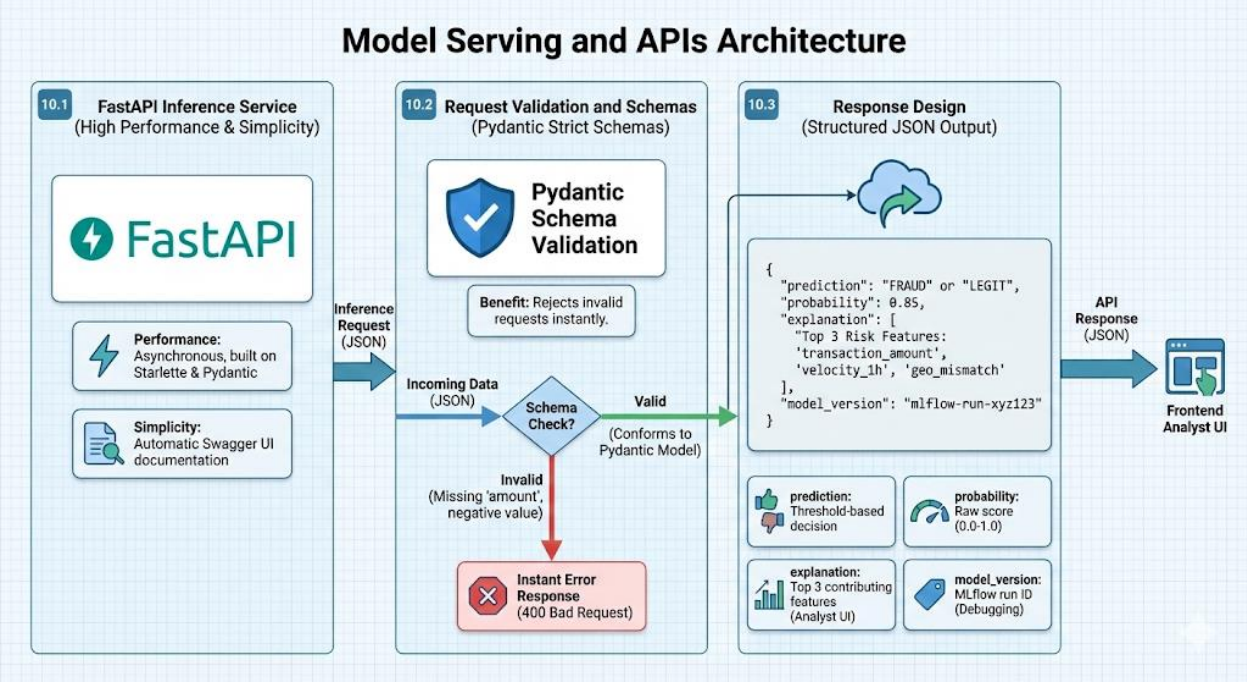
9.3 Model Promotion Workflow

The **Model Registry** governs the lifecycle:

© Whitebox Learning, Inc. 2026

1. **Staging:** A successful training run is registered to "Staging."
2. **Validation:** Automated tests run against the Staging model (integration tests, latency checks).
3. **Production:** Only after manual sign-off (or passing strict automated gates) is a model promoted to "Production."

10. Model Serving and APIs



10.1 FastAPI Inference Service

10.1.1 Why We Did That

- **Performance:** FastAPI is asynchronous and highly performant (built on Starlette/Pydantic).
- **Simplicity:** It automatically generates interactive documentation (Swagger UI), which helped frontend teams integrate faster.

© Whitebox Learning, Inc. 2026

10.2 Request Validation and Schemas

- We used **Pydantic** models to define strict schemas for the input JSON.
- **Benefit:** Invalid requests (e.g., missing "amount" or negative values) are rejected instantly with clear error messages *before* reaching the model logic.

10.3 Response Design

The API returns a structured JSON object:

- prediction: "FRAUD" or "LEGIT" (based on a threshold).
- probability: The raw score (0.0 to 1.0).
- explanation: The top 3 features contributing to the risk score (for the analyst UI).
- model_version: The MLflow run ID (for debugging).

11. CI/CD for ML Systems

11.1 Training Pipelines

- **Trigger:** A scheduled job (weekly) or a code merge to the main branch.
- **Action:** Spins up a Docker container, pulls data via DVC, trains the model, and logs results to MLflow.

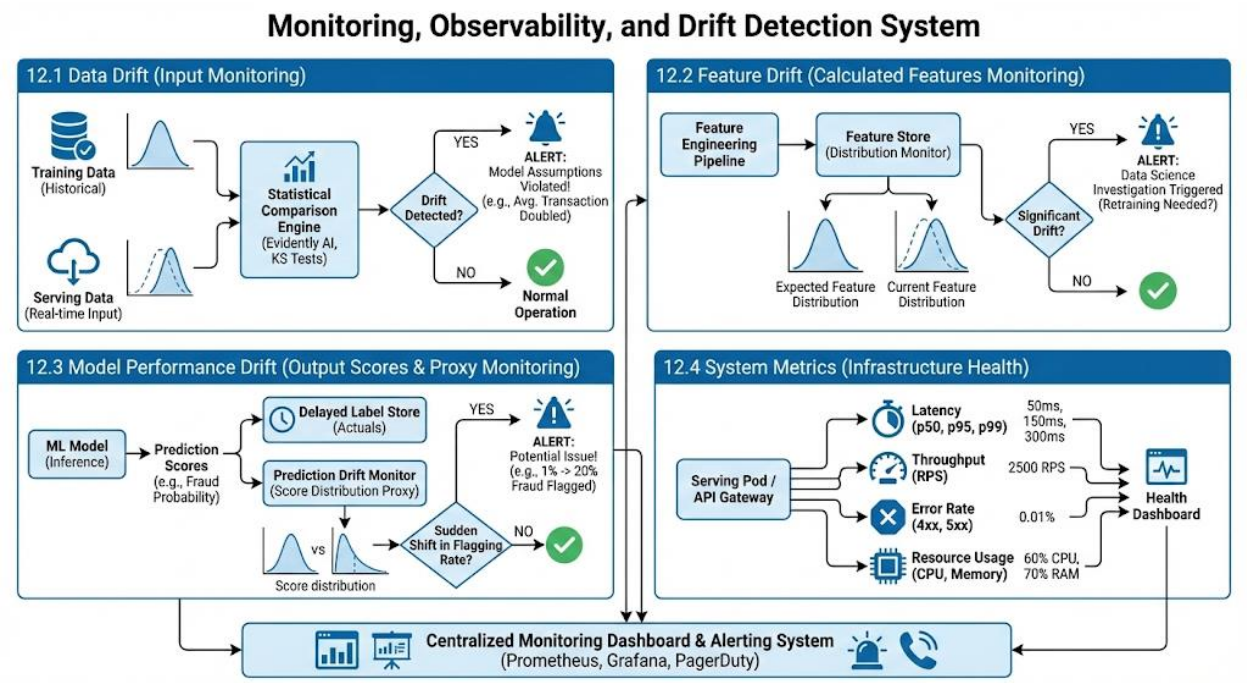
11.2 Deployment Pipelines

- **Trigger:** A model promotion event in the Registry.
- **Action:**
    - Builds a new Docker image containing the FastAPI app and the specific model artifact.

© Whitebox Learning, Inc. 2026

- Pushes the image to **Amazon ECR**.
- Updates the Kubernetes deployment manifest to reference the new image tag.

12. Monitoring, Observability, and Drift Detection



12.1 Data Drift

- **What:** Monitoring the statistical properties of input data.
- **Example:** If the "Average Transaction Amount" suddenly doubles (e.g., due to inflation or a currency bug), the model's assumptions are violated.
- **Tool:** We use **Evidently AI** or custom Kolmogorov-Smirnov (KS) tests to compare the *serving* distribution vs. the *training* distribution.

© Whitebox Learning, Inc. 2026

12.2 Feature Drift

- **What:** Monitoring the distribution of the *calculated* features.
- **Action:** Significant drift triggers an alert to the data science team to investigate if retraining is needed.

12.3 Model Performance Drift

- **What:** Comparing predictions against actuals (when labels arrive).
- **Challenge:** Since labels are delayed, we monitor **Prediction Drift** (the distribution of output scores) as a proxy. If the model suddenly starts flagging 20% of traffic as fraud (vs. the usual 1%), something is wrong.

12.4 System Metrics

- **Latency:** p50, p95, and p99 response times.
- **Throughput:** Requests per second (RPS).
- **Error Rate:** 4xx and 5xx HTTP errors.
- **Resource Usage:** CPU and Memory utilization of the pods.

13. Governance, Security, and Compliance

13.1 Explainability

- For Logistic Regression, we provide **Global Explainability** (feature weights) and **Local Explainability** (contribution of each feature to a specific score).
- This satisfies the "Right to Explanation" requirements (e.g., GDPR).

© Whitebox Learning, Inc. 2026

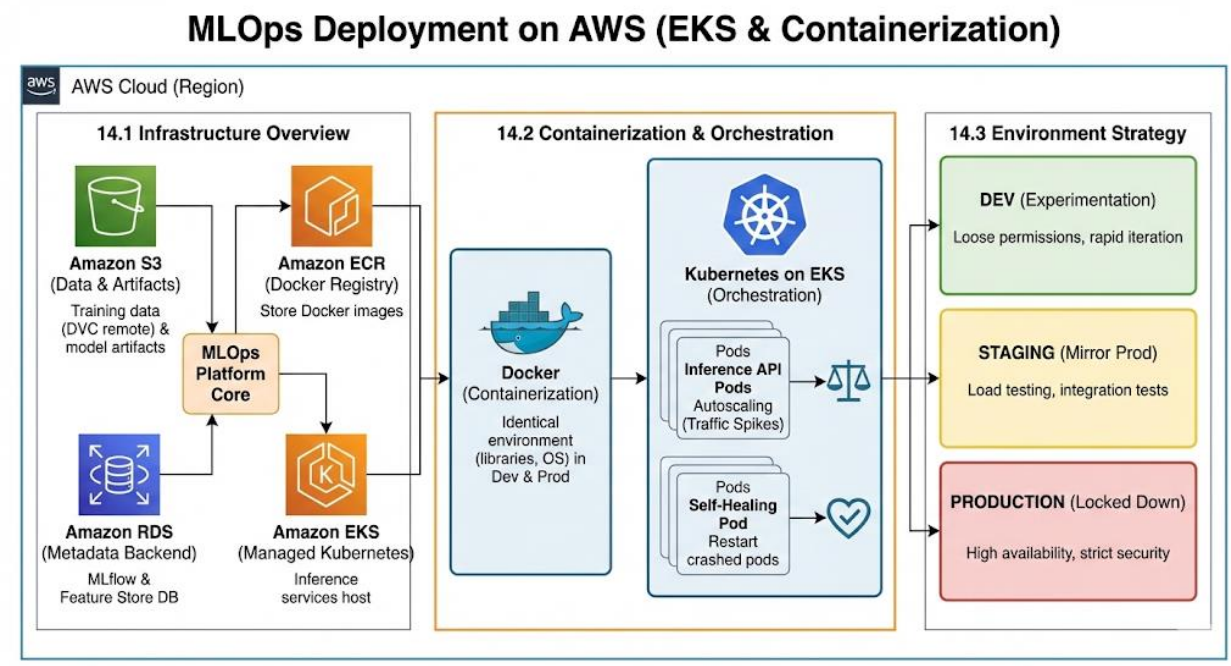
13.2 Audit Trails

- Every decision is logged. We can trace a decision back to the specific API request, the model version used, the data that trained that model, and the code commit that defined the architecture.

13.3 Access Control

- **RBAC:** Role-Based Access Control limits who can deploy to Production. Data Scientists have "Read" access to Prod but "Write" access only to Dev/Staging.

14. Deployment on AWS



14.1 Infrastructure Overview

- **Amazon S3:** Stores training data (DVC remote) and model artifacts.

© Whitebox Learning, Inc. 2026

- **Amazon ECR:** Registry for Docker images.
- **Amazon EKS:** Managed Kubernetes for running the inference services.
- **Amazon RDS:** Database backend for MLflow and the Feature Store.

14.2 Containerization and Orchestration

- **Docker:** Ensures the environment (libraries, OS dependencies) is identical in Dev and Prod.
- **Kubernetes (EKS):** Handles **Autoscaling** (adding more pods during traffic spikes) and **Self-Healing** (restarting crashed pods).

14.3 Environment Strategy

- **Dev:** For experimentation. Loose permissions.
- **Staging:** Mirror of Production. Used for load testing and integration tests.
- **Production:** Locked down. High availability.

15. Phase Two: Advanced Modeling (Future State)

15.1 Motivation for Phase Two

As we collect more data and engineer richer features, we expect to hit a "performance ceiling" with Logistic Regression. Fraudsters often use complex, non-linear patterns (e.g., specific combinations of time, location, and amount) that linear models struggle to detect.

15.2 XGBoost Model Design

- **Algorithm: XGBoost (Gradient Boosted Trees).**
- **Advantage:** Naturally handles non-linear interactions and complex decision boundaries.

© Whitebox Learning, Inc. 2026

- **Explainability:** We will migrate to **SHAP (SHapley Additive exPlanations)** values to maintain explainability with this more complex model.

15.3 Comparison with Logistic Regression

Feature	Phase 1: Logistic Regression	Phase 2: XGBoost
Complexity	Low (Linear)	High (Non-Linear)
Interpretability	High (Coefficients)	Medium (Requires SHAP)
Training Time	Fast	Slower
Inference Latency	Ultra-Fast (<10ms)	Slower (~20-50ms)
Performance	Good Baseline	Superior Accuracy

16. Future Enhancements

- **Automated Retraining:** Triggering training pipelines automatically when drift is detected.
- **Real-Time Feature Expansion:** Adding streaming features (e.g., clickstream data) via Kafka.
- **Ensemble Strategies:** Running Logistic Regression and XGBoost in parallel and averaging their scores for robust decision-making.
- **Graph Neural Networks:** Exploring graph-based features to detect fraud rings and organized crime networks.

17. References

[1] Neptune.ai, “What Is MLOps?”, Neptune.ai Blog, 2023

© Whitebox Learning, Inc. 2026

https://github.com/sarwarbeing-ai/Agentic_Design_Patterns/blob/main/Agentic_Design_Patterns.pdf

© Whitebox Learning, Inc. 2026